CSCI6730 – Operating Systems

Project #1: Multi-process and multi-thread

Due date: 11:59pm, 9/21/2024

Description

In this project, you will design and implement matrix multiplication programs. You are provided with a single-process version, which you will extend to incorporate both multi-process and multi-threaded architectures.

Preparation: Download and prepare the project file

Download the project file (OS_Proj1.tar.gz) from ELC. You need to copy that into "odin" server using any ssh or sftp program.

- Once you have the tarball in odin server, you can unzip it with the following command:

```
% tar xzvf OS_Proj1.tar.gz
```

- Compile the program with "make" command.

```
% cd OS_Proj1
% make
```

- **Use vcf0~3 servers for this project**

```
% ssh vcf0.cs.uga.edu
```

Part 0: Matrix Multiplication Program

1. The project folder contains the following source code files.
   - *matmul.c*: The main program file responsible for initialization, reading matrices, executing matrix multiplication (single, multi-process, and multi-thread), performance measurement, result output, and result comparison. It also includes the single-process version of matrix multiplication (*mat_mult_single*()).
   - *matmul_core.c*: Contains utility functions.
   - *matmul.h*: A header file that defines functions and global variables.
   - *matmul_multi_proc.c*: A skeleton file for implementing the multi-process architecture, which you will complete in Part 1.
   - *matmul_multi_thread.c*: A skeleton file for implementing the multi-threaded architecture, which you will complete in Part 2.

- *matgen.c*: A standalone program that generates matrices of a specified size. You can use this to generate various matrices for evaluating your solution.
- *Makefile*: A file used for compiling the program with the make command.

2. "*matmul*" program accepts the following command-line arguments.
   - (required) 1st argument: input file that contains two matrices
   - (optional) 2nd argument: crash rate (default is 0%)
   - Example: *./matmul ./inputs/mat1.txt*

3. The input file should be structured as follows:
   - [value of m] [value of n] [value of p]
   - [Matrix A]
   - [Matrix B]
   - For instance, mat1.txt file contains A[4*3] and B[3*4], the input file looks like the following.

```
sh-4.2$ cat matrices/mat1.txt
4 3 4

1 2 3
4 5 6
7 8 9
1 2 3

9 8 7 6
5 4 3 2
1 9 8 7
```

4. You can create input files manually to test the correctness and performance of your implementation, or you can generate them randomly using ***./matgen*** program.
   - ./matgen <m> <n> <p> → generates m*n and n*p matrices
   - The generated matrices will be displayed on the screen. To save the output to a file, use the following command:
   - ./matgen 4 3 4 > ./matrices/test.txt

5. We multiply two matrices, A and B, and store the result into a matrix C
   - (input) Size of A: m * n
   - (input) Size of B: n * p
   - (result) Size of C : m * p
   - Wikipedia - Matrix multiplication:

If $\mathbf{A}$ is an $m \times n$ matrix and $\mathbf{B}$ is an $n \times p$ matrix,

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} b_{11} & b_{12} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{np} \end{pmatrix}$$

the *matrix product* $\mathbf{C} = \mathbf{AB}$ (denoted without multiplication signs or dots) is defined to be the $m \times p$ matrix

$$\mathbf{C} = \begin{pmatrix} c_{11} & c_{12} & \cdots & c_{1p} \\ c_{21} & c_{22} & \cdots & c_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & \cdots & c_{mp} \end{pmatrix}$$

such that

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj} = \sum_{k=1}^{n} a_{ik}b_{kj},$$

for $i = 1, \ldots, m$ and $j = 1, \ldots, p$.

6. "matmul.c" program has three parts:
   - **Part 1:** Allocate and initialize matrix data structures, and read the input file. We use six matrices:
     i. Input matrices: A[m][n], B[n][p]
     ii. Transposed matrix: B_tran[p][n]
        1. For easier calculation, we use a transposed matrix of B, which flips a matrix over its diagonal.
     iii. Three output matrices:
        1. C_single[m][p]: store the result of the single process version.
        2. C_multi_proc[m][p]: store the result of the multi-process version.
        3. C_multi_thread[m][p]: store the result of the multi-threaded version.
   - **Part 2:** Execute three versions of matrix multiplication (single-process, multi-process, multi-threaded) while measuring their execution times. The single-process version is already implemented in the *mat_mult_single()* function, and you will implement the multi-process and multi-threaded versions.
   - **Part 3:** Print the result matrices and compare them. The results from all three versions should match.

Part 1: Multi-process Matrix Multiplication Program (20%)

The main limitation of the single-process program is its lack of scalability. To overcome this, you will implement the matrix multiplication program to use a multi-process model. The main process will create multiple child processes and efficiently distribute the workload among them. Each child process will compute its portion of the matrix multiplication and send the result back to the main process using an inter-process communication (IPC) channel. You are free to choose any IPC mechanism for this project. The main process will wait for all child processes to finish and then collect their results via IPC.

You will implement `mat_mult_multi_proc(int numProc)` function in "matmul_multi_proc.c" file.

1. You will modify "matmul_multi_proc.c" to support a multi-process model.
2. The main process creates "numProc" processes (equal to the value of "m") and distributes the work among these child processes.
3. The main body of the child process should be implemented in the `child_process_core()` function.
4. After creating the child processes, the main process should wait for them to complete and gather their results via IPC. The final results should be stored in "C_multi_proc".

Part 2: Multi-thread Matrix Multiplication Program (20%)

To further improve performance, we need to implement a multi-threaded version of the matrix multiplication program. Unlike the multi-process model, the multi-threaded approach allows threads to share memory space, eliminating the need for inter-process communication. Each thread can directly access the global output matrix, making this model more efficient in terms of resource usage and communication overhead. The main thread will create multiple child threads to divide the workload, and each thread will compute a portion of the matrix multiplication.

You will implement `mat_mult_multi_thread(int numThread)` function in "matmul_multi_thread.c" file.

1. Modify "matmul_multi_thread.c" to support a multi-threaded model.
2. The main thread creates "numThread" threads (equal to the value of "m") and divides the job among them.
3. The main body of the child thread should be implemented in `child_thread_core()` function.
4. After creating the child threads, the main thread will wait for them to finish. The final results should be stored in "C_multi_thread".

Part 3: Performance Evaluation (30%)

Performance evaluation is important in understanding the scalability, efficiency, and real-world applicability of different matrix multiplication approaches. By analyzing performance, you can identify bottlenecks, optimize resource usage, and determine

which model—single-process, multi-process, or multi-threaded—performs best under varying conditions. Additionally, evaluating the impact of different inter-process communication (IPC) methods in the multi-process model will provide further insight into optimization strategies.

1. Plan your evaluation with various matrix sizes, paying special attention to the value of "m" (the number of child processes/threads), as it might significantly impact performance.
2. Measure and compare the performance of all three versions (single-process, multi-process, and multi-threaded).
3. Analyze the results and, if necessary, design additional tests to further evaluate performance.
4. Document the full details of your evaluation design, observations, and analysis in your report. Try to explain the reasons behind the performance outcomes. If there are any unexplained results, include them in the report for further discussion.

Part 4: Recovery from the crash (30%)

n a real-world system, processes or threads may crash before completing their assigned tasks, leading to incorrect final outputs. In this project, if one or more child processes or threads crash before completing their work (either before sending results through a pipe or before storing them in memory), the main process or thread must handle these failures. The goal is to monitor each child, and if any crash, the main process/thread should create new child processes/threads to complete the job.

- You can use 2nd command-line arguments (an integer between 0 and 30) to trigger crash. A value of 30 means each child has 30% chance of being terminated before completing the task. 0 means no chance of a crash.
    - In the multi-process model, child processes crash by calling the *abort()* function.
    - In the multi-threaded model, child threads crash by calling *pthread_exit(NULL)*.
- (optional) For multi-process version, you can use **`WIFEXITED`** and **`WIFSIGNALED`** macros to check the exit status of each child. You can find more details from the linux manual page. (man 2 wait)
- We do NOT consider crashes of the parent process or thread.
- Any child can crash if the given crash rate is > 0. Your design should be able to handle recursive crashes. For instance, consider the following case:
    - The parent creates 10 child processes/threads. 4 of them crash.
    - The parent creates 4 additional processes/threads to cover the crashed ones. But 2 of them crashed again.
    - The parent creates 2 more, but 1 of them crashes.
    - The parent creates 1 more child process, and it finishes jobs.

  Your solution should be resilient to these failures and ensure that the job is completed correctly, even in the presence of multiple crashes.

Part 5: Project Report

Your project report should provide a detailed description of your design and implementation for each part of the project. You don't need to explain your code line by line; instead, focus on the high-level design and reasoning behind your approach.

- **Parts 1 and 2:** Describe your design and implementation of the multi-process and multi-threaded matrix multiplication models. Focus on the structure of your solution and how you approached dividing the workload among processes and threads.
- **Part 3:** Discuss your performance evaluation strategy. Describe how you designed the evaluation, present your observations, and provide an analysis of the results. Include any further discussion regarding unexpected results or additional insights gained from the evaluation.
- **Part 4:** Outline the high-level design of your crash recovery strategy. Explain how your implementation handles crashes in various scenarios and ensures that the job is completed even in the presence of multiple child process/thread failures.

Only "pdf" format will be accepted.

Submission

Prepare a tarball file using the following command

%tar czvf p1.tar.gz report.pdf Makefile matmul_multi_proc.c matmul_multi_thread.c

(include other files if you have modified them).

1. Your report file must include your 8xx number and name.
2. **Ensure that your code compiles and runs correctly on the vcf0 machine.**
    a. The code will be evaluated on vcf0 machine.
    b. If your code fails to compile on vcf0, you will receive 0 points.
3. Submit the tarball (p1.tar.gz) through ELC.